

WEB APPLICATION PENETRATION TESTING REPORT

Mini Project — Ethical Hacking Externship
Elewayte

Target Application:

<http://testphp.vulnweb.com/>

Prepared by: Pawan Kumar V

Date: July 2026

Classification: Confidential — Educational Use Only

Authorization & Scope Notice

This assessment was conducted exclusively against <http://testphp.vulnweb.com/>, an intentionally vulnerable web application built and hosted by Acunetix specifically for security training, tool testing, and educational penetration testing practice. Acunetix explicitly authorizes security testing against this application. No other systems were tested, and no destructive actions (data deletion, denial-of-service) were performed against the target.

This report was prepared as a mini-project deliverable for the Elewayte Ethical Hacking externship program.

Executive Summary

This report documents the findings of a web application penetration test performed against the publicly available, intentionally vulnerable test site testphp.vulnweb.com. The assessment focused on manual, black-box testing of the application's input handling across its search, authentication, product-browsing, and guestbook features.

Five vulnerabilities were identified and are documented in this report, spanning SQL Injection (including an authentication bypass), Reflected and Stored Cross-Site Scripting (XSS), and an Open Redirect. The application's core weakness is a consistent lack of input validation and output encoding across nearly every user-controllable parameter — a pattern that, in a real production system, would allow an attacker to fully compromise the underlying database and its users with minimal effort.

Severity breakdown: 2 Critical · 2 High · 1 Medium

Scope

- Target: <http://testphp.vulnweb.com/> (all publicly reachable pages and parameters)
- Testing type: Black-box, unauthenticated and authenticated (using the site's own test account) manual testing
- Out of scope: Any infrastructure not directly part of the web application (hosting provider, network layer, third-party CDNs)

Methodology

Testing followed a standard black-box web application methodology consistent with the OWASP Testing Guide: reconnaissance and site mapping, input parameter identification, manual injection testing of each parameter, verification of impact, and severity rating using a CVSS-informed qualitative scale (Critical / High / Medium / Low).

Finding 1: SQL Injection (UNION-Based) — artists.php

Severity: **Critical**

Vulnerable URL: `http://testphp.vulnweb.com/artists.php?artist=1`

Vulnerable Parameter: artist (GET)

Payload: `artist=1'` and `artist=-1 UNION SELECT 1,2,3--`

Description

The artist parameter on artists.php is concatenated directly into a backend SQL query without sanitization or parameterization. Submitting a single quote as the parameter value breaks the query's syntax and causes the application to return a raw MySQL error, confirming the injection point. From there, a UNION-based technique can be used to append an attacker-controlled query to the original one and read arbitrary data back through the page's normal output.

Proof of Concept — Observation Slides

1. Navigate to `http://testphp.vulnweb.com/artists.php?artist=1` — the page loads normally, displaying information for artist ID 1.
2. Append a single quote: `http://testphp.vulnweb.com/artists.php?artist=1'` — the page returns a MySQL syntax error instead of the artist page, confirming unsanitized input reaches the SQL query.
3. Determine the column count using ORDER BY: increment `artist=1 order by 1`, then `order by 2`, `order by 3`, `order by 4` — the query stops erroring at 3 and fails again at 4, confirming the underlying query selects exactly 3 columns.
4. Confirm which columns are reflected in the output: `http://testphp.vulnweb.com/artists.php?artist=-1 UNION SELECT 1,2,3--` (using `-1` ensures the original query returns no rows, so only the injected UNION row is visible).
5. Extract database metadata by replacing the reflected columns with useful queries, e.g. `artist=-1 UNION SELECT 1,version(),database()--` to reveal the MySQL version and active database name.
6. Enumerate table names: `artist=-1 UNION SELECT 1,group_concat(table_name),3 FROM information_schema.tables WHERE table_schema=database()--` returns the full list of application tables, including a users table.
7. Extract credentials: `artist=-1 UNION SELECT 1,group_concat(uname,0x3a,pass),3 FROM users--` returns usernames and passwords directly in the page output.

Business Impact

This is a full database compromise vector. An attacker can read every table in the application's database — including usernames, passwords, and (on this specific demo app) simulated credit card data — without needing any valid credentials. In a real e-commerce or customer-facing application, this would constitute a complete breach of customer PII and payment data, triggering regulatory notification obligations (e.g., under GDPR or India's DPDP Act), significant financial and reputational damage, and potential downstream fraud against affected customers.

Remediation

- Replace all dynamic SQL string concatenation with parameterized queries / prepared statements (e.g., PDO with bound parameters in PHP, or an ORM that parameterizes by default).
- Apply the principle of least privilege to the database account used by the application — it should not have permission to read tables outside what the application strictly needs.
- Disable verbose database error messages in production; log them server-side only and return a generic error page to the user.
- Add a Web Application Firewall (WAF) as a defense-in-depth layer, not as the primary fix.

References

- OWASP – SQL Injection: https://owasp.org/www-community/attacks/SQL_Injection
- OWASP Cheat Sheet – SQL Injection Prevention: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- PortSwigger Web Security Academy – SQL Injection: <https://portswigger.net/web-security/sql-injection>

Finding 2: SQL Injection — Authentication Bypass (login.php / userinfo.php)

Severity: **Critical**

Vulnerable URL: <http://testphp.vulnweb.com/userinfo.php>

Vulnerable Parameters: uname, pass (POST)

Payload: uname=' OR '1'='1 | pass=' OR '1'='1

Description

The login form builds its SQL query by directly concatenating the submitted username and password into a WHERE clause, without parameterization. Submitting a payload that makes the WHERE clause unconditionally true allows an attacker to log in as the first user in the users table without knowing any valid password.

Proof of Concept — Observation Slides

1. Navigate to the login page and open the browser's network/developer tools to observe the login POST request.
2. In the username field, enter: ' OR '1'='1 — and in the password field, enter the same: ' OR '1'='1
3. Submit the form.
4. Observe that the application responds with a successful login and redirects to the user's account page, despite no valid credentials having been supplied.
5. Inspect the response cookies — note that a login cookie is set containing account information for the bypassed user.

Business Impact

Authentication bypass defeats the application's core access control. An attacker gains access to another user's account — and in many real applications, the first row returned is often an administrative or seed account, escalating this into full administrative compromise. Combined with Finding 1, an attacker does not even need this bypass to read data directly, but this finding demonstrates that the application's authentication logic itself provides zero effective protection, which is a critical finding independent of the UNION-based read access.

Remediation

- Use parameterized queries for all authentication logic — never build authentication SQL by string concatenation.
- Hash passwords with a strong, salted algorithm (bcrypt/argon2) and compare hashes server-side rather than including password values in SQL WHERE clauses at all.
- Implement account lockout / rate limiting on the login endpoint to slow down automated bypass or brute-force attempts.
- Log and alert on authentication queries that return unexpected result-set shapes.

References

- OWASP – Authentication Bypass via SQL Injection: https://owasp.org/www-community/attacks/SQL_Injection
- OWASP Cheat Sheet – Authentication: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

Finding 3: Reflected Cross-Site Scripting — search.php

Severity: High

Vulnerable URL: <http://testphp.vulnweb.com/search.php?test=query>

Vulnerable Parameter: searchFor (POST)

Payload: `<script>alert('XSS')</script>`

Description

The search feature echoes the submitted search term back into the page ('searched for: ...') without HTML-encoding it. Because the value is inserted directly into the page's HTML rather than being encoded first, a script tag submitted as the search term is rendered and executed by the browser rather than displayed as plain text.

Proof of Concept — Observation Slides

1. Navigate to <http://testphp.vulnweb.com/search.php>
2. In the search box, enter the payload: `<script>alert('XSS')</script>`
3. Click the search / go button.
4. Observe a JavaScript alert box pop up in the browser, confirming the injected script executed.
5. View the page source and locate the unescaped `<script>` tag inside the 'searched for' heading to confirm the root cause.

Business Impact

Reflected XSS requires tricking a victim into clicking a crafted link, but once triggered it executes arbitrary JavaScript in the victim's authenticated session. An attacker could craft a link that exfiltrates the victim's session cookie to an attacker-controlled server, effectively hijacking their session without ever knowing their password. Given this application also stores credentials in a cookie (a separate but related weakness), a successful reflected XSS attack here could directly expose a victim's plaintext password.

Remediation

- HTML-encode all user-controlled output using context-appropriate encoding (e.g., `htmlspecialchars()` in PHP with `ENT_QUOTES`) before rendering it back to the page.
- Implement a Content-Security-Policy header (e.g., `script-src 'self'`) so injected inline scripts are blocked by the browser even if an encoding gap is missed.
- Set the `HttpOnly` flag on session cookies so JavaScript (including injected XSS payloads) cannot read them.

References

- OWASP – Cross-Site Scripting (XSS): <https://owasp.org/www-community/attacks/xss/>
- OWASP Cheat Sheet – XSS Prevention: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- PortSwigger Web Security Academy – Reflected XSS: <https://portswigger.net/web-security/cross-site-scripting/reflected>

Finding 4: Stored Cross-Site Scripting — guestbook.php

Severity: High

Vulnerable URL: <http://testphp.vulnweb.com/guestbook.php>

Vulnerable Parameter: text (POST, guestbook message field)

Payload: `<script>alert('Stored XSS')</script>`

Description

The guestbook feature saves visitor-submitted messages to the database and renders them, unescaped, to every subsequent visitor of the guestbook page. Unlike the reflected XSS in Finding 3, this payload persists — it does not require a victim to click a special link at all; simply visiting the guestbook page is enough to trigger it.

Proof of Concept — Observation Slides

1. Navigate to <http://testphp.vulnweb.com/guestbook.php>
2. In the message field, enter the payload: `<script>alert('Stored XSS')</script>` and submit the entry.
3. Reload the guestbook page (or revisit it in a fresh browser session).
4. Observe the JavaScript alert fires automatically on page load, with no further action required — confirming the payload was stored and is now served to every visitor.

Business Impact

Stored XSS is significantly more dangerous than reflected XSS because it requires no social engineering — every visitor to the affected page is automatically attacked. An attacker could use this to silently harvest session cookies from every guestbook visitor, redirect users to phishing pages, or deploy an in-browser cryptomining script, all without needing to distribute a malicious link.

Remediation

- Apply strict output encoding to all stored user content at render time, not just at input time (encoding on output is the authoritative control).
- Strip or reject HTML tags entirely from fields that are not meant to contain formatting, such as a guestbook message.
- Apply the same Content-Security-Policy and HttpOnly cookie protections recommended in Finding 3, since they mitigate both reflected and stored XSS.

References

- OWASP – Stored XSS: <https://owasp.org/www-community/attacks/xss/>
- PortSwigger Web Security Academy – Stored XSS: <https://portswigger.net/web-security/cross-site-scripting/stored>

Finding 5: Open Redirect — redir.php

Severity: Medium

Vulnerable URL: `http://testphp.vulnweb.com/redir.php?r=<url>`

Vulnerable Parameter: r (GET)

Payload: `r=http://evil-example.com`

Description

The redir.php endpoint takes a URL supplied in the r parameter and issues an HTTP redirect to it, without validating that the destination belongs to an approved/whitelisted domain. Because the redirect originates from the trusted testphp.vulnweb.com domain, the resulting link looks trustworthy to a user even though its final destination is entirely attacker-controlled.

Proof of Concept — Observation Slides

1. Construct the URL: `http://testphp.vulnweb.com/redir.php?r=http://evil-example.com`
2. Open the URL in a browser (or send a HEAD request and inspect the response headers).
3. Observe an HTTP 302 redirect response with a Location header pointing to the attacker-supplied domain.
4. Confirm the browser is transparently sent to the external site, despite the initial link showing the trusted testphp.vulnweb.com domain.

Business Impact

Open redirects are a common building block in phishing campaigns: attackers send victims a link that appears to originate from a trusted domain, increasing click-through and trust, before silently forwarding them to a credential-harvesting or malware-hosting page. This can also be abused to bypass naive URL allow-lists used by email security gateways or to steal OAuth tokens in certain misconfigured authentication flows.

Remediation

- Maintain a server-side whitelist of allowed redirect destinations, and reject any r value not on that list.
- Where a whitelist is impractical, use indirect redirect tokens (e.g., a short-lived ID mapped server-side to a destination) rather than accepting a raw URL from the client.
- Display an interstitial 'you are leaving this site' warning page for any redirect to an external domain.

References

- OWASP – Unvalidated Redirects and Forwards:
https://owasp.org/www-community/attacks/Unvalidated_Redirects_and_Forwards

Summary of Findings

#	Finding	Location	Severity	Category
1	UNION-based SQL Injection	artists.php?artist=	Critical	Injection
2	SQLi Authentication Bypass	userinfo.php (login)	Critical	Broken Auth
3	Reflected XSS	search.php	High	XSS
4	Stored XSS	guestbook.php	High	XSS
5	Open Redirect	redir.php?r=	Medium	Unvalidated Redirect

Overall Recommendations

1. Adopt parameterized queries / prepared statements universally — this single fix eliminates both SQL injection findings (1 and 2), which are the highest-severity issues in this report.
2. Apply output encoding to all user-controlled data before rendering it in HTML — this eliminates both XSS findings (3 and 4).
3. Implement a Content-Security-Policy header as defense-in-depth against any XSS gaps missed by encoding fixes.
4. Validate and whitelist redirect destinations to close the open redirect (Finding 5).
5. Set HttpOnly and Secure flags on all session cookies, and avoid storing credentials in cookies at all.
6. Re-test the application after remediation to confirm each fix, ideally using both manual verification and an automated scanner as a regression check.

Conclusion

The testphp.vulnweb.com application, though intentionally vulnerable for training purposes, illustrates a pattern seen repeatedly in real-world web applications: a single root cause — trusting user input without validation or encoding — cascades into multiple, severe vulnerability classes. The findings in this report, if present in a production system, would together allow an attacker to fully compromise the application's data and its users. Prioritizing the SQL injection findings first, followed by the XSS findings, would eliminate the most severe risk with the least remediation effort.

References (Consolidated)

- OWASP Top 10: <https://owasp.org/www-project-top-ten/>
- OWASP Testing Guide: <https://owasp.org/www-project-web-security-testing-guide/>
- OWASP Cheat Sheet Series: <https://cheatsheetseries.owasp.org/>
- PortSwigger Web Security Academy: <https://portswigger.net/web-security>
- Acunetix — Vulnerable Test Site (testphp.vulnweb.com), official Acunetix demo application for security testing practice